

Decision Tree (AI)

Aim

To implement a Decision Tree classifier to predict whether a game of tennis will be played, based on weather attributes such as Outlook, Temperature, Humidity, and Wind.

Objectives

- To understand decision tree construction using entropy/information gain.
- To encode categorical attributes for model training.
- To train a Decision Tree classifier on the given dataset.
- To predict the class label for a new, unseen instance.

Algorithm

- 1 Start.
- 2 Collect the training dataset with attributes and class labels (PlayTennis).
- 3 Encode categorical attribute values into numeric form.
- 4 Select the attribute that gives the highest information gain (lowest entropy) as the root.
- 5 Split the dataset based on the selected attribute's values.
- 6 Repeat the splitting process recursively for each branch.
- 7 Stop splitting when all instances in a branch belong to one class, or no attributes remain.
- 8 Use the trained tree to classify a new sample.
- 9 Display the tree rules and the prediction.
- 10 Stop.

Source Code (Python)

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.preprocessing import LabelEncoder
import pandas as pd
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny', 'Sunny',
               'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild', 'Cool', 'Mild', 'Mild',
                   'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'Normal',
                'Normal', 'High', 'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong',
            'Strong', 'Strong', 'Weak', 'Strong'],
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
}
df = pd.DataFrame(data)
print("Training Dataset:")
print(df.to_string(index=False))
encoders = {}
df_enc = df.copy()
for col in df.columns:
```

```

le = LabelEncoder()
df_enc[col] = le.fit_transform(df[col])
encoders[col] = le
X = df_enc.drop('PlayTennis', axis=1)
y = df_enc['PlayTennis']
clf = DecisionTreeClassifier(criterion='entropy', random_state=0)
clf.fit(X, y)
print("\nDecision Tree Rules:")
print(export_text(clf, feature_names=list(X.columns)))
sample = [['Sunny', 'Cool', 'High', 'Strong']]
sample_enc = pd.DataFrame([[encoders[col].transform([val])[0] for col, val in zip(X.columns,
sample[0])]], columns=X.columns)
pred = clf.predict(sample_enc)
result = encoders['PlayTennis'].inverse_transform(pred)
print(f"Prediction for {sample[0]} -> PlayTennis = {result[0]}")

```

Output

```

Training Dataset:
  Outlook Temperature Humidity   Wind PlayTennis
   Sunny         Hot      High   Weak         No
   Sunny         Hot      High Strong         No
Overcast         Hot      High   Weak         Yes
   Rain         Mild      High   Weak         Yes
   Rain         Cool    Normal   Weak         Yes
   Rain         Cool    Normal Strong         No
Overcast         Cool    Normal Strong         Yes
   Sunny         Mild      High   Weak         No
   Sunny         Cool    Normal   Weak         Yes
   Rain         Mild    Normal   Weak         Yes
   Sunny         Mild    Normal Strong         Yes
Overcast         Mild      High Strong         Yes
Overcast         Hot     Normal   Weak         Yes
   Rain         Mild      High Strong         No

Decision Tree Rules:
|--- Outlook <= 0.50
|   |--- class: 1
|--- Outlook > 0.50
|   |--- Humidity <= 0.50
|   |   |--- Outlook <= 1.50
|   |   |   |--- Wind <= 0.50
|   |   |   |   |--- class: 0
|   |   |   |   |--- Wind > 0.50
|   |   |   |   |--- class: 1
|   |   |--- Outlook > 1.50
|   |   |   |--- class: 0
|   |--- Humidity > 0.50
|   |   |--- Wind <= 0.50
|   |   |   |--- Temperature <= 1.00
|   |   |   |   |--- class: 0
|   |   |   |   |--- Temperature > 1.00
|   |   |   |   |--- class: 1
|   |   |--- Wind > 0.50
|   |   |   |--- class: 1
Prediction for ['Sunny', 'Cool', 'High', 'Strong'] -> PlayTennis = No

```

Result

The Decision Tree classifier was successfully trained on the PlayTennis dataset. For the sample (Sunny, Cool, High, Strong), the model predicted PlayTennis = No.